EXECUTIVE OVERVIEW · MANAGEMENT

RoslynGate

Less context, more precision: how to make an AI agent understand a real .NET codebase without reading all of it.

A semantic layer between AI agents and .NET solutions. Built for teams with large repositories, legacy projects, and code that does not forgive an invented reference.

01 — THE PROBLEM

The agent sees text. The business needs certainty.

When an agent searches for symbols with grep, it finds matches. When code has overloads, partial classes, interfaces, generated code, and legacy projects, a match is not an answer.

1 Expensive context

Reading entire files to find one reference consumes tokens, time, and model attention.

2 Fragile answers

A false positive can lead to editing the wrong file. A false negative can hide real impact.

3 Large solutions

Ninety-nine projects and legacy packages are not a lab case; they are the daily reality.

4 Generic tools

grep is excellent for text. It does not understand symbols. An IDE understands symbols but does not feed an agent over MCP.

> *The opportunity is not “another search tool”. It is giving the agent the minimum semantic answer so it does not have to read half the repository.*

02 — THE PROPOSAL

A semantic layer for .NET agents

RoslynGate loads the solution with MSBuild and answers questions with Roslyn: definition, references, hierarchy, and name search. The agent receives compact JSON with file and line.

Precision

Resolves symbols, not text matches. Distinguishes overloads, types, and projects.

Economy

Returns the minimum: location, use kind, and actionable pagination.

Integration

Speaks MCP. Claude Code and other agents use it without custom integrations.

What the team gets

BEFORE	WITH ROSLYNGATE
“Search for ‘Doors’ and inspect files”	“Resolve the symbol and return its semantic references.”
The agent reads entire files	The agent receives `file:line` and reads only when needed.
Overloads resolved by trial and error	Candidates or parameter types.
Results that depend on the prompt	Stable order, pagination, and actionable errors.

> The binary is self-contained: the runtime is bundled. The .NET SDK is still required because MSBuild uses it to read projects. RoslynGate honors the SDK pinned by `global.json` .

03 — EVIDENCE

The saving was measured, not estimated

Two A/B tests with Claude Code on a 99-project legacy solution. Same prompt, same model, with and without RoslynGate.

33.9%
fewer tokens · 3 symbols

21.2%
fewer tokens · 6 symbols

30%
lower cost · 3 symbols

22%
lower cost · 6 symbols

QUERY	TOKENS WITH	TOKENS WITHOUT	DURATION WITH	DURATION WITHOUT
3 symbols	94,158	142,506	88.1 s	29.2 s
6 symbols	103,904	131,865	102.3 s	61.8 s

How to read it

- The saving is real in tokens and cost, not just characters.
- Latency is worse because each new process loads MSBuild; an interactive session keeps the workspace alive.
- With six symbols RoslynGate returned more references than the baseline: it was not truncation.
- This is a sample; the percentage depends on the query and repository.

04 — DIFFERENTIATION

It does not compete with grep. It competes with ambiguity.

grep will remain the right tool for free text. RoslynGate covers the question grep cannot answer: “what symbol is this, and who really uses it?”.

TOOL	STRENGTH	LIMIT FOR AN AGENT
grep / rg	Fast, simple, cheap.	Text, not semantics.
IDE / LSP	Excellent for a person.	Not an MCP contract for agents.
Analyzers	Code-quality rules.	Do not answer ad-hoc queries.
Global indexes	Broad coverage.	Do not use the repository's real compilation.
RoslynGate	Semantic .NET, compact, read-only.	Needs the SDK and a fresh snapshot.

> *The differentiation is not “faster than grep”. It is “the correct answer for a symbol, in the format an agent can consume”.*

Why now

Agents already write and edit code. The bottleneck is no longer generating text; it is understanding the repository precisely. That is RoslynGate's space.

05 — ADOPTION

How it enters a .NET team

Adoption is incremental. It does not require changing the compiler or the build flow; it requires installing the package, having the SDK, and configuring MCP.

1 Install

`npm` or `bun`. The wrapper downloads the self-contained binary and verifies its checksum.

2 Configure

`roslyngate setup claude --solution . --skill` creates `.mcp.json` and a project skill.

3 Measure

An A/B per representative prompt with `claude-ab.ps1`. No number is promised without measuring it.

4 Scale

Incremental watcher, explicit reload, project filters, and impact analysis.

Risks and mitigation

RISK	MITIGATION
Missing or different SDK	Preflight, respect for `global.json`, actionable message.
Stale workspace after edits	Incremental watcher, visible `stale`, and `reload_workspace`.
Initial latency	Prewarm, persistent session, and a separate benchmark.
Exaggerated expectations	Publish results with sample size, cost, and limits.

06 — HORIZON

From tool to platform

The base is already layered and exposes a contract. That allows growth into higher-value cases without rewriting the engine.

Live freshness

Edit and query without restarting; explicit reload only for structural changes.

Overload precision

Resolve by parameter types and cut agent round trips.

Change impact

Which implementations and calls break before touching an interface.

CI and review

Semantic verification in pipelines and PRs, not only in chat.

Architecture

Real dependency maps between projects and layers.

Teams

Shared skill, reproducible setup, and per-prompt metrics.

> *The goal is not for the agent to read less just to read less. It is for the agent to read the right thing.*

READ-ONLY · SEMANTIC · MCP · MEASURABLE · SCALABLE